TASK 3 – DBMS

Seller and Inventory Management System

1. Aim

To design and implement a MySQL-based Seller and Inventory Management System that organizes seller details, electronic product records, warehouse stock, and reorder information.

2. Objectives

• Create a structured database for sellers, products, and warehouse stock.

• Maintain seller contact and business information.

• Store product category, price, and seller details.

• Track available quantity and minimum reorder level.

• Identify products that require restocking.

• Perform insertion, updating, deletion, and retrieval operations.

• Use SQL joins and aggregate functions to generate useful reports.

3. Database Design

The redesigned system contains three main tables:

Seller Table
• Seller ID
• Seller Name
• City
• Contact Number

Product Table
• Product ID
• Product Name
• Category
• Unit Price
• Seller ID

Inventory Table
• Stock ID
• Product ID
• Quantity
• Reorder Level
• Stock Status

Relationship: Seller → Product → Inventory

One seller can provide several products, and each product has a corresponding inventory record.

Seller and Inventory Management System

4. Create Database

```
CREATE DATABASE store_inventory_db;
USE store_inventory_db;
```

5. Create Seller Table

```
CREATE TABLE Seller (
    seller_id INT PRIMARY KEY,
    seller_name VARCHAR(60),
    city VARCHAR(40),
    phone VARCHAR(15)
);
```

6. Create Product Table

```
CREATE TABLE Product (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(60),
    category VARCHAR(40),
    unit_price DECIMAL(10,2),
    seller_id INT,
    FOREIGN KEY (seller_id) REFERENCES Seller(seller_id)
);
```

7. Create Inventory Table

```
CREATE TABLE Inventory (
    stock_id INT PRIMARY KEY,
    product_id INT,
    quantity INT,
    reorder_level INT,
    stock_status VARCHAR(20),
    FOREIGN KEY (product_id) REFERENCES Product(product_id)
);
```

Primary keys uniquely identify records, while foreign keys connect products with sellers and inventory.

Seller and Inventory Management System

8. Insert Seller Records

```
INSERT INTO Seller VALUES
(101, 'Nova Digital Supplies', 'Chennai', '9001122334'),
(102, 'Bright Office Hub', 'Coimbatore', '9012233445'),
(103, 'Metro Gadget House', 'Madurai', '9023344556'),
(104, 'Prime Tech Stores', 'Salem', '9034455667');
```

9. Insert Product Records

```
INSERT INTO Product VALUES
(301, 'Laser Keyboard', 'Accessories', 1250.00, 101),
(302, 'USB Microphone', 'Audio', 2450.00, 102),
(303, 'Wi-Fi Router', 'Networking', 3200.00, 103),
(304, 'LED Monitor 24', 'Display', 8900.00, 104),
(305, 'Wireless Mouse', 'Accessories', 850.00, 101);
```

10. Insert Inventory Records

```
INSERT INTO Inventory VALUES
(501, 301, 25, 10, 'In Stock'),
(502, 302, 7, 8, 'Reorder'),
(503, 303, 18, 5, 'In Stock'),
(504, 304, 3, 5, 'Reorder'),
(505, 305, 40, 15, 'In Stock');
```

The records use different sellers, products, IDs, prices, and stock levels from the reference version.

Seller and Inventory Management System

11. Display All Sellers

```
SELECT * FROM Seller;
```

12. Display Products with Price

```
SELECT product_name, category, unit_price
FROM Product;
```

13. Display Products with Seller Names

```
SELECT Seller.seller_name, Product.product_name, Product.unit_price
FROM Seller
INNER JOIN Product ON Seller.seller_id = Product.seller_id;
```

14. Display Products Requiring Reorder

```
SELECT Product.product_name, Inventory.quantity, Inventory.reorder_level
FROM Product
INNER JOIN Inventory ON Product.product_id = Inventory.product_id
WHERE Inventory.stock_status = 'Reorder';
```

15. Display Products with Stock Above 20

```
SELECT Product.product_name, Inventory.quantity
FROM Product
INNER JOIN Inventory ON Product.product_id = Inventory.product_id
WHERE Inventory.quantity > 20;
```

Seller and Inventory Management System

16. Generate Complete Inventory Report

```
SELECT Product.product_id, Product.product_name, Product.category,
       Inventory.quantity, Inventory.reorder_level, Inventory.stock_status
FROM Product
INNER JOIN Inventory ON Product.product_id = Inventory.product_id;
```

17. Calculate Total Inventory Value

```
SELECT SUM(Product.unit_price * Inventory.quantity)
       AS total_inventory_value
FROM Product
INNER JOIN Inventory ON Product.product_id = Inventory.product_id;
```

18. Update Stock Quantity

For example, after receiving new wireless mouse stock:

```
UPDATE Inventory
SET quantity = 55, stock_status = 'In Stock'
WHERE product_id = 305;
```

19. Delete a Product Record

The related inventory record is deleted first to maintain the foreign-key relationship.

```
DELETE FROM Inventory WHERE product_id = 304;
DELETE FROM Product WHERE product_id = 304;
```

These queries demonstrate filtering, joining, aggregation, updating, and deletion.

Seller and Inventory Management System

20. Sample Output

```
Product Name        Quantity   Reorder Level   Status
-------------------------------------------------------
Laser Keyboard       25         10              In Stock
USB Microphone       7          8               Reorder
Wi-Fi Router         18         5               In Stock
LED Monitor 24       3          5               Reorder
Wireless Mouse       40         15              In Stock
```

21. Conclusion

The Seller and Inventory Management System provides a structured approach to managing suppliers, products, and warehouse stock. The redesigned database uses primary and foreign keys to maintain relationships between the tables. SQL commands can add new records, retrieve product information, identify items that need restocking, calculate inventory value, update stock quantities, and remove unwanted records. The system therefore supports efficient inventory monitoring and provides useful information for day-to-day store management.

Key Features

• Seller and product information is stored in separate tables.

• Inventory quantities are connected to individual products.

• Reorder levels help identify products that need attention.

• JOIN queries combine related information from multiple tables.

• Aggregate SQL functions can calculate overall inventory value.